Exercise 1 — Build it from a prompt

Build working software from a deliberately incomplete brief

What you are building

A small piece of financial software: a way for an investor to sell shares of stock they hold, and see the money and profit that sale produced. Nothing more — the point is what happens when you build *even something this small* from requirements that only look complete.

The requirements

Notes from a conversation with the product owner. This is everything you get.

> *Our investors hold stock in their portfolios, and they need to be able to sell it.*
>
> *When an investor sells, tell them how much money the sale brought in and what profit they made on it. The profit matters — they have to report it to the tax authority, so it needs to be right.*
>
> *There is one complication. We buy the same stock more than once, at different times and at different prices. So the cost of the shares being sold depends on which shares you consider sold.*
>
> *The money from a sale goes into the investor's cash balance.*

Your test investor is **Alice**. She bought 10 shares of AAPL at 100.00, and later another 5 shares at 120.00. Today's market price for AAPL is 150.00.

Nowhere does it say **which** shares count as sold first — and that choice changes the profit number that ends up on somebody's tax return. That is not a mistake in this document: real requirements are incomplete like this all the time.

Pick a language

Java and Python are the two most common choices, but use whichever language you're comfortable with — the exercise doesn't depend on it. Pick one, then use the matching block below.

Java

- Java 21, Maven, a standalone project with its own `pom.xml`.
- JUnit 5 for tests.
- Every monetary amount as `BigDecimal` — never `double` or `float`.

Python

- Python 3.11+, a standalone project (`pyproject.toml` or `requirements.txt`).
- Tests with the built-in `unittest` module, or whatever test tooling is already approved and installable in your environment — don't assume a specific third-party test library is available.
- Every monetary amount as `decimal.Decimal` — never a plain `float`.

Another language — same idea: an idiomatic project, a real test framework, and an exact (not floating-point) representation for money.

The suggested prompt

Copy this to your AI assistant, filling in the technical block for your language. Tweak the wording if you like, but don't add information the brief above doesn't contain — and don't tell the assistant which accounting method to use; that decision is what this exercise is about.

SUGGESTED PROMPT

```
Build a small piece of software for selling stock from an investor's
portfolio.

Investors hold stock, bought at different times and possibly at different
prices. When an investor sells shares, tell them how much money the sale
brought in and what profit they made. The profit needs to be exactly right,
because it gets reported to a tax authority. The cost of the shares being sold
depends on which shares you consider sold — the same stock may have been
bought more than once, at different prices. The money from a sale goes into
the investor's cash balance.

To try it out, use this example: an investor named Alice bought 10 shares of
AAPL at 100.00, and later 5 more shares at 120.00. Today's market price for
AAPL is 150.00.

[ Paste in the technical block for your chosen language here. ]

Write tests for this as you build it.
```

Talk to the assistant as much as you like after that. What matters is that you never write down a fuller specification, acceptance criteria, or a class diagram — anything it needs to know, you say out loud in the conversation.

Build it

Go back and forth with the assistant until:

1. the code builds;
2. the tests it wrote pass; and

3. you can sell some of Alice's shares and see a proceeds figure, a profit figure, and an updated cash balance.

Before you trust that green build

Stop here, before you move on, and answer honestly:

> *Which accounting method did your assistant pick — did you tell it to, or did it decide on its own?*

There are at least three defensible answers to "which shares count as sold": oldest first (FIFO), blended average, or newest first (LIFO) — all real accounting methods. Your assistant picked one *silently*, since the brief never named one.

More importantly: **the assistant wrote the code from its own reading of the brief, then wrote the tests from that same reading.** The tests never checked against a business rule — there was none to check against. A test suite that only has to agree with the implementation that produced it will always pass. The same blind spot hits anywhere else the brief stayed quiet: selling zero shares, more than you hold, a negative price.

Score yourself

Run each check below against your own implementation and record what actually happens — not what you believe should happen. If your code can't answer a question at all, that's itself an answer: write "cannot tell". Throughout, **Alice's portfolio** is 10 shares of AAPL at 100.00, then 5 more at 120.00 — fifteen shares, cash balance 0.00, market price 150.00 unless stated otherwise.

Before you check anything, write down:

> *What did I never tell the assistant, that it had to decide on its own?*

Part A — The money

#	CHECK	REQUIRED RESULT	YOURS
A1	Sell 12 shares at 150.00	proceeds 1800.00, cost basis **1240.00**, profit **560.00**	
A2	Sell 8 shares at 150.00	proceeds 1200.00, cost basis **800.00**, profit **400.00**	
A3	Sell 1 share at 150.00	proceeds 150.00, cost basis 100.00, profit 50.00	
A4	Sell 10 shares at 150.00	proceeds 1500.00, cost basis 1000.00, profit 500.00	

#	CHECK	REQUIRED RESULT	YOURS
A5	Sell all 15 shares at 150.00	proceeds 2250.00, cost basis 1600.00, profit 650.00	
A6	Sell 8 shares at 90.00	proceeds 720.00, cost basis 800.00, profit **−80.00** — a loss is valid, not an error	
A7	After selling 8, what is left	one lot of **2** shares at 100.00 and one of **5** at 120.00	
A8	After selling 12, what is left	**3** shares at 120.00; the first purchase is gone entirely	

A1 is the one that matters. The required method is **FIFO**: the shares sold are the oldest ones you still hold. If your cost basis for 12 shares came out as anything else, your assistant chose a different accounting method — and it chose it without telling you.

WHAT YOUR CODE DID	COST BASIS ON 12	PROFIT
FIFO — oldest shares first	**1240.00**	**560.00**
Weighted average cost	1280.00	520.00
LIFO — newest shares first	1300.00	500.00

All three are recognised cost-basis methods. Which of them you are *allowed* to use depends on the jurisdiction, the instrument and the account type — a question for a tax specialist, not for this exercise. Two of them are the wrong answer *here*, and the brief never said which.

Part B — The refusals

#	CHECK	REQUIRED RESULT	YOURS
B1	Sell 0 shares	Refused. Not “a sale of nothing for nothing”.	
B2	Sell −5 shares	Refused. Check carefully — many implementations quietly *increase* the position and *reduce* the cash balance instead.	
B3	Sell 16 shares of the 15 held	Refused, with an error that says how many were available and how many were asked for	
B4	Sell 5 shares of MSFT, which Alice does not hold	Refused, and distinguishably from B3 — “you don’t hold this” is not the same problem as “you don’t hold enough”	
B5	Sell at a price of 0.00 or a negative price	Refused	
B6	Ticker `"aapl"`, `"TOOLONG"`, `"123"`, `""`	Refused. A ticker is 1–5 uppercase letters.	

Part C — What happens after a refusal

#	CHECK	REQUIRED RESULT	YOURS
C1	After the refused sale of 16 shares, inspect the position	Untouched: 15 shares, still 10 at 100.00 and 5 at 120.00	
C2	After that same refusal, inspect the cash balance	Untouched: 0.00	

If your implementation consumed part of the oldest lot before discovering it could not finish, it failed halfway through and left the portfolio in a state that never should have existed.

Part D — The parts nobody mentions

#	CHECK	REQUIRED RESULT	YOURS
D1	What type holds money in your code?	`BigDecimal` / `Decimal`. If it's `double`, `float`, or a plain binary float type, you have a defect — see below.	
D2	Sell 8 shares, then print the profit exactly as your code stores it	An exact 400.00	
D3	Sell all 15 shares, then try to sell 1 more	Refused, and the position no longer exists	
D4	Sell all 15, then buy 4 more at 130.00	A fresh position of exactly 4 shares — no ghosts of the old one	

On D1 and D2. An implementation that stores money as a binary float and uses weighted-average cost will compute the profit on an 8-share sale as `proceeds - averageCost * 8`, producing something like `346.66666666666663` — an inexact value, its precise digits depending on the order of operations. Now look at the test your assistant wrote for that number: it almost certainly reads something like `assertEquals(346.67, profit, 0.01)`, with a tolerance. The tolerance exists because floating-point money doesn't come out exact — and it's the reason the test passed without you ever finding out.

Scoring

Count Part A as one point per row, Parts B, C and D as one point per row — twenty points in total. A low score isn't failure: some of these answers were genuinely unknowable from the brief alone. What matters is noticing which ones you got right by design, and which by luck.

What’s next

This exercise handed you a brief and asked you to talk your way to working software. Exercise 2 hands you the same kind of problem again — but this time the behaviour lives in versioned specification files that exist **before** any code does, reviewable line by line instead of buried in a conversation only you saw.